Java

Builder, Buffer, string methods, stack & heap, class and object intro

```java
public class Main
{
        public static void main(String[] args) {
                System.out.println("Hello World");
                //string ->
//              data types => primitive and non-primitive

        // primitive => fixed size eg : int -> 4 bytes
        // non-primitive => no fixed size eg: object ,array , string

        // memory => stack and heap

        // heap => garbage collector
        // String literal
        String name ="fiit";
        String name1="FIIT";

        System.out.println(name == name1);

        String name2= new String("FIIT");

        System.out.println(name == name2);
        System.out.println(name.equals(name2));

        // String is immutable => not changeable


        //string methods
        System.out.println(name.length());
        System.out.println(name.charAt(1));

        System.out.println(name.indexOf("I"));

        System.out.println(name.lastIndexOf('I'));

        System.out.println(name.indexOf('I'));

        System.out.println(name.toUpperCase());

        String upperCase = name.toUpperCase();

        System.out.println(upperCase);
```

```java
        System.out.println(name.toLowerCase());

        String spaceTrim = "  Hello    World   ";

        System.out.println(spaceTrim.trim());

        System.out.println(name.equalsIgnoreCase(name1));


        System.out.println(name.contains("fi"));

        // substring (begin, end) => 0, 4

        System.out.println(spaceTrim.substring(0,4));

        System.out.println(spaceTrim.replace(' ', '@'));

        System.out.println(name.startsWith("fi"));
        System.out.println(name.endsWith("it"));

        System.out.println(name.concat(" Academy"));


        // StringBuilder and StringBuffer => mutable we can modify the value


        // StringBuilder => mutable, non-synchronized , single-threaded it
performs faster

        StringBuilder sb = new StringBuilder("Hello");

        sb.append(" World");
        sb.insert(4 ,"@");

        sb.delete(4, 6);
        sb.replace(4,6, "Jav");

        // System.out.println(sb.reverse());

        System.out.println(sb);



        // stringbuffer => mutable , synchronized (thread safety) ,
multithreaded environment
        StringBuffer sbb = new StringBuffer("String");
```

```java
        sbb.append(" World");
        sbb.insert(4 ,"@");

        sbb.delete(4, 6);
        sbb.replace(4,6, "Jav");


        System.out.println(sbb);


        }
}
```

```java
 class App{

    int data = 10 ; // field in class

    //method in class
    public void display(String name){
        System.out.println(name);
    }
}

public class Main
{
        public static void main(String[] args) {
                System.out.println("Hello World");

                //oops => object oriented programming

//              it's a programming paradigm organizes software around object

        // class  => blueprint  => doesn't create memory
        // object => instance of class => create memory for that object


        App obj = new App();

        System.out.println(obj.data);

        obj.display("FIIT");

        }
}
```

Theory Notes

1. Strings in Java

- **Immutable** → once created, cannot be changed.
- Stored in **String Constant Pool** when created as literals.
- Common methods:
    - length() → returns length.
    - charAt(index) → returns character at position.
    - indexOf() / lastIndexOf() → find position of character/substring.
    - toUpperCase() / toLowerCase() → case conversion.
    - trim() → removes leading/trailing spaces.
    - equals() → compares values.
    - equalsIgnoreCase() → compares values ignoring case.
    - startsWith() / endsWith() → check prefixes/suffixes.
    - concat() → joins strings.
    - substring(begin, end) → extracts part of string.
    - replace() → replaces characters.

2. StringBuilder vs StringBuffer

- Both are **mutable** (values can be changed).
- **StringBuilder**:
    - Faster, not synchronized.
    - Best for single-threaded applications.
- **StringBuffer**:
    - Synchronized (thread-safe).
    - Best for multi-threaded applications.
- Common methods:
    - append() → add text.
    - insert() → insert text at position.
    - delete(begin, end) → remove characters.
    - replace(begin, end, str) → replace substring.
    - reverse() → reverse content.
    -

3. OOP Basics

- **Class** → blueprint (no memory until object is created).
- **Object** → instance of a class (creates memory).
- **Fields** → variables inside a class.
- **Methods** → functions inside a class.
- Example:

```
class App {
   int data = 10;
   public void display(String name) {
      System.out.println(name);
   }
}
```

Usage:

```
App obj = new App();
System.out.println(obj.data);
obj.display("FIIT");
```

4. Memory Management

- **Stack** → stores primitive data and references.
- **Heap** → stores objects.
- **Garbage Collector** → automatically frees unused objects.

Practice Tasks

Short Answer

1. Why are Strings immutable in Java?
2. What is the difference between equals() and == for Strings?
3. When should you use StringBuilder instead of StringBuffer?
4. What is the role of the garbage collector in Java?

Coding Exercises

1. Use StringBuilder to reverse a string entered by the user.
2. Demonstrate the difference between equals() and == with string literals and objects.

3. Create a class Student with fields id, name, and a method display(). Instantiate and print details.
4. Use StringBuffer to build a thread-safe string manipulation example.
5. Create a method that trims spaces and replaces them with underscores in a given string.